ЗАНЯТИЕ 9

Определить структурный тип, описывающий расписание маршрутов электропоездов (название маршрута, пункт отправления, пункт прибытия, время отправления, время прибытия, количество вагонов, тип электропоезда: товарный, пассажирский, ремонтный и др.). Заполнить структурный массив 20-ю записями. Переписать из исходного массива в другой массив, информацию только о тех пассажирских маршрутах, что идут в туристические города. Затем новый массив отсортировать по пункту отправления вагонов. Вывести все данные по конкретному электропоезду. Вывести 3 самых коротких пассажирских поезда. Реализовать функцию изменения данных о маршруте. Реализовать вывод отфильтрованных данных в виде оберточной функции.

КОД

```
#include <iostream>
#include <cstring>
using namespace std;



const int SIZE = 20;
enum tip { tovar, passa, remont };
char tipname[3][25] = { "товарный", "пассажирский", "ремонтный" };
struct ttime {
   int hour, minuts;
};
struct train {
   char marshryt[30];
   char pynkt_otpav[30];
   char pynkt_pribit[30];
   ttime time_otprav;
```

```
        ttime time_pribit;
        int count_vagon;
        tip tip_train;
    };

    train* trains;

    train* init() {
        train* trains = new train[SIZE];
        trains[0] = { "Lastochka-701", "Москва", "Суздаль", {3, 45}, {11, 40}, 8,
passa };
        trains[1] = { "Gryzovoy-115", "Екатеринбург", "Тюмень", {23,10},
{5,20}, 45, tovar };
        trains[2] = { "Express-202", "Санкт-Петербург", "Тверь", {14,30},
{18,45}, 10, passa };
        trains[3] = { "Tavria", "Москва", "Ялта", {17,20}, {10,15}, 14, passa };
        trains[4] = { "Tex-09", "Депо Люблино", "Депо Подольск", {1,0},
{2,15}, 2, remont };
        trains[5] = { "Lastochka-803", "Адлер", "Сочи", {8,5}, {8,45}, 5, passa };
        trains[6] = { "Kometa", "Санкт-Петербург", "Петергоф", {10,0},
{10,45}, 6, passa };
        trains[7] = { "Ygolniy-44", "Новокузнецк", "Магнитогорск", {6,40},
{22,15}, 60, tovar };
        trains[8] = { "Tatarstan", "Москва", "Казань", {19,8}, {6,30}, 12, passa };
        trains[9] = { "Sapsan-761", "Москва", "Нижний Новгород", {16,30},
{20,15}, 10, passa };
        trains[10] = { "VladimirskiyExpress", "Москва", "Владимир", {7,45},
{9,30}, 7, passa };
        trains[11] = { "Alektrichka-611", "Тула", "Серпухов", {13,20}, {14,35},
6, passa };
```

```
        trains[12] = { "Lastochka-787", "Санкт-Петербург", "Выборг", {9,30}, {10,45}, 5, passa };
        trains[13] = { "RuskealskiyExpress", "Санкт-Петербург", "Сортавала", {18,30}, {22,50}, 7, passa };
        trains[14] = { "Kontenerniy-78", "Владивосток", "Хабаровск", {12,15}, {8,40}, 71, tovar };
        trains[15] = { "Pskovitianka", "Москва", "Псков", {20,23}, {8,30}, 9, passa };
        trains[16] = { "Pyteec-2", "Депо Орехово", "Депо Железнодорожная", {9,0}, {11,30}, 1, remont };
        trains[17] = { "Yantarniy", "Калининград", "Зеленоградск", {8,40}, {9,15}, 4, passa };
        trains[18] = { "Express-317", "Ростов-на-Дону", "Краснодар", {15,20}, {18,50}, 11, passa };
        trains[19] = { "Kavkaz", "Москва", "Кисловодск", {8,25}, {9,40}, 13, passa };
        return trains;
    }

    void printTrain(const train& t) {
        cout << "Маршрут: " << t.marshryt << endl;
        cout << "Отправление: " << t.pynkt_otpav << " в ";
        cout << (t.time_otprav.hour < 10 ? "0" : "") << t.time_otprav.hour << ":";
        cout << (t.time_otprav.minuts < 10 ? "0" : "") << t.time_otprav.minuts << endl;
        cout << "Прибытие: " << t.pynkt_pribit << " в ";
        cout << (t.time_pribit.hour < 10 ? "0" : "") << t.time_pribit.hour << ":";
        cout << (t.time_pribit.minuts < 10 ? "0" : "") << t.time_pribit.minuts << endl;
        cout << "Вагонов: " << t.count_vagon << endl;
```

```
    cout << "Тип: " << tipname[t.tip_train] << endl;
}

//Вывести все данные по конкретному электропоезду.
void display() {

    char name[30];

    cout << "\nВведите название маршрута: ";
    cin >> name;

    for (int t = 0; t < SIZE; t++) {
        if (!strcmp(name, trains[t].marshryt)) {
            printTrain(trains[t]);
            return;
        }
    }
    cout << "Маршрут не найден.\n\n";
}

//Реализовать функцию изменения данных о маршруте.
void input(int i) {

    int tip_t;

    cout << "Название маршрута: ";
    cin >> trains[i].marshryt;

    cout << "Пункт отправления: ";
    cin >> trains[i].pynkt_otpav;
```

```
    cout << "Пункт прибытия: ";
    cin >> trains[i].pynkt_pribit;

    cout << "Время отправления (час минуты): ";
    cin >> trains[i].time_otprav.hour >> trains[i].time_otprav.minuts;

    cout << "Время прибытия (час минуты): ";
    cin >> trains[i].time_pribit.hour >> trains[i].time_pribit.minuts;

    cout << "Количество вагонов: ";
    cin >> trains[i].count_vagon;

    cout << "Тип поезда (0-товарный, 1-пассажирский, 2-ремонтный): ";
    cin >> tip_t;
    trains[i].tip_train = (tip)tip_t;
}

void update() {

    int i;
    char name[30];

    cout << "\nВведите наименование маршрута: ";
    cin >> name;

    for (i = 0; i < SIZE; i++)
        if (!strcmp(name, trains[i].marshryt)) break;

    if (i == SIZE) {
```

```
            cout << "Маршрут не найден.\n";
            return;
        }

        cout << "Введите новую информацию о маршруте";
        input(i);

        cout << "\nИнформация успешно обновлена!\n";
        printTrain(trains[i]);
    }
    //Реализовать вывод отфильтрованных данных в виде оберточной функции.
    void obertka(const train* arr, int size) {
        for (int t = 0; t < size; t++) {
            printTrain(arr[t]);
            cout << "\n";
        }
    }
    //Переписать из исходного массива в другой массив, информацию только о тех пассажирскихмаршрутах, что идут в туристические города.
    //Затем новый массив отсортировать по пункту отправления вагонов.
    int tyrist_train( train* result) {
        const char* tyrist[] = { "Суздаль", "Ялта", "Сочи", "Петергоф", "Казань", "Владимир", "Выборг", "Сортавала(Рускеала)", "Псков", "Зеленоградск", "Кисловодск" };
        train nova[20] = {};
        int nova_count = 0;
        train pass_train[20] = {};
        int count_pass_train = 0;
```

```
// Выборка только пассажирских поездов
for (int i = 0; i < SIZE; i++) {
    if (trains[i].tip_train == passa) {
        pass_train[count_pass_train] = trains[i];
        count_pass_train++;
    }
}

// Переписать из исходного массива в другой массив, информацию только о тех пассажирских маршрутах, что идут в туристические города.
for (int i = 0; i < count_pass_train; i++) {
    for (int j = 0; j < int(sizeof(tyrist) / sizeof(tyrist[0])); j++) {
        if (strcmp(pass_train[i].pynkt_pribit, tyrist[j]) == 0) {
            nova[nova_count] = pass_train[i];
            nova_count++;
        }
    }
}

//  Затем новый массив отсортировать по пункту отправления вагонов.
for (int i = 0; i < nova_count - 1; i++) {
    for (int j = 0; j < nova_count - i - 1; j++) {
        if (strcmp(nova[j].pynkt_otpav, nova[j + 1].pynkt_otpav) > 0) {
            train temp = nova[j];
            nova[j] = nova[j + 1];
            nova[j + 1] = temp;
        }
    }
}
```

```
    for (int i = 0; i < nova_count; i++) {
        result[i] = nova[i];
    }

    return nova_count;
}
//Вывести 3 самых коротких пассажирских поезда.
void short_train() {

    train pass_train[20] = {};
    int count_pass_train = 0;

    for (int i = 0; i < SIZE; i++) {
        if (trains[i].tip_train == passa) {
            pass_train[count_pass_train] = trains[i];
            count_pass_train++;
        }
    }
    // Сортировка по количеству вагонов
    for (int i = 0; i < count_pass_train; i++) {
        for (int j = 0; j < count_pass_train - 1; j++) {
            if (pass_train[j].count_vagon > pass_train[j + 1].count_vagon) {
                train temp = pass_train[j];
                pass_train[j] = pass_train[j + 1];
                pass_train[j + 1] = temp;
            }
        }
    }
    // Вывод трех самых коротких пассажирских поездов
    cout << "\n\nТри самых коротких пассажирских поезда" << endl;
```

```
    obertka(pass_train, 3);
    cout << "\n";
}

int main() {

    setlocale(LC_ALL, "");

    trains = init();

    train nova[20] = {};
    int nova_count = 0;

    cout << "Поиск конкретного электропоезда";
    display();

    cout << "\nИзменение данных о маршруте";
    update();

    cout << "\nТуристические поезда";
    nova_count = tyrist_train(nova);
    cout << "\nОтсортированный список пассажирских поездов, следующих в туристические города\n";
    obertka(nova, nova_count);

    short_train();

    delete[] trains;
    return 0;
```

}

РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

```
Поиск конкретного электропоезда
Введите название маршрута: Lastochka-701
Маршрут: Lastochka-701
Отправление: Москва в 03:45
Прибытие: Суздаль в 11:40
Вагонов: 8
Тип: пассажирский

Изменение данных о маршруте
Введите наименование маршрута: dgfb
Маршрут не найден.

Туристические поезда
Отсортированный список пассажирских поездов, следующих в туристические города
Маршрут: Lastochka-803
Отправление: Адлер в 08:05
Прибытие: Сочи в 08:45
Вагонов: 5
Тип: пассажирский

Маршрут: Yantarniy
Отправление: Калининград в 08:40
Прибытие: Зеленоградск в 09:15
Вагонов: 4
Тип: пассажирский

Маршрут: Lastochka-701
Отправление: Москва в 03:45
Прибытие: Суздаль в 11:40
Вагонов: 8
Тип: пассажирский

Маршрут: Tavria
Отправление: Москва в 17:20
Прибытие: Ялта в 10:15
Вагонов: 14
Тип: пассажирский

Маршрут: Tatarstan
Отправление: Москва в 19:08
Прибытие: Казань в 06:30
Вагонов: 12
Тип: пассажирский
```

```
Маршрут: VladimirskiyExpress
Отправление: Москва в 07:45
Прибытие: Владимир в 09:30
Вагонов: 7
Тип: пассажирский

Маршрут: Pskovitianka
Отправление: Москва в 20:23
Прибытие: Псков в 08:30
Вагонов: 9
Тип: пассажирский

Маршрут: Kavkaz
Отправление: Москва в 08:25
Прибытие: Кисловодск в 09:40
Вагонов: 13
Тип: пассажирский

Маршрут: Kometa
Отправление: Санкт-Петербург в 10:00
Прибытие: Петергоф в 10:45
Вагонов: 6
Тип: пассажирский

Маршрут: Lastochka-787
Отправление: Санкт-Петербург в 09:30
Прибытие: Выборг в 10:45
Вагонов: 5
Тип: пассажирский
```

```
Три самых коротких пассажирских поезда
Маршрут: Yantarniy
Отправление: Калининград в 08:40
Прибытие: Зеленоградск в 09:15
Вагонов: 4
Тип: пассажирский

Маршрут: Lastochka-803
Отправление: Адлер в 08:05
Прибытие: Сочи в 08:45
Вагонов: 5
Тип: пассажирский

Маршрут: Lastochka-787
Отправление: Санкт-Петербург в 09:30
Прибытие: Выборг в 10:45
Вагонов: 5
Тип: пассажирский
```

ЗАНЯТИЕ 10

Задание 1. Считать из текстового файла данные числового поля для вашей.

Например, исходный файл имеет 2 столбца: ФИО и пол (0 – мужской, 1 - женский). Из файла считываются данные и если фамилия совпадает с очередным элементом массива структур, то в поле pol записывается соответствующее значение из файла.

```
Ivanov 0
Ivanova 1
Petrov 0
Sidorova 1
Makarkin 0
```

Задание 2. Сделать чтение / запись своей структуры в бинарный файл. Оформить в виде подпрограмм.

КОД

```
#include <iostream>
#include <cstring>
#include <fstream>
using namespace std;



const int SIZE = 20;
enum tip { tovar, passa, remont };
char tipname[3][25] = { "товарный", "пассажирский", "ремонтный" };
struct ttime {
   int hour, minuts;
};
struct train {
   char marshryt[30];
   char pynkt_otpav[30];
   char pynkt_pribit[30];
   ttime time_otprav;
```

```
        ttime time_pribit;
        int count_vagon;
        tip tip_train;
    };

    train* trains;

    train* init() {
        train* trains = new train[SIZE];
        trains[0] = { "Lastochka-701", "Москва", "Суздаль", {3, 45}, {11, 40}, 8,
passa };
        trains[1] = { "Gryzovoy-115", "Екатеринбург", "Тюмень", {23,10},
{5,20}, 45, tovar };
        trains[2] = { "Express-202", "Санкт-Петербург", "Тверь", {14,30},
{18,45}, 10, passa };
        trains[3] = { "Tavria", "Москва", "Ялта", {17,20}, {10,15}, 14, passa };
        trains[4] = { "Tex-09", "Депо Люблино", "Депо Подольск", {1,0},
{2,15}, 2, remont };
        trains[5] = { "Lastochka-803", "Адлер", "Сочи", {8,5}, {8,45}, 5, passa };
        trains[6] = { "Kometa", "Санкт-Петербург", "Петергоф", {10,0},
{10,45}, 6, passa };
        trains[7] = { "Ygolniy-44", "Новокузнецк", "Магнитогорск", {6,40},
{22,15}, 60, tovar };
        trains[8] = { "Tatarstan", "Москва", "Казань", {19,8}, {6,30}, 12, passa };
        trains[9] = { "Sapsan-761", "Москва", "Нижний Новгород", {16,30},
{20,15}, 10, passa };
        trains[10] = { "VladimirskiyExpress", "Москва", "Владимир", {7,45},
{9,30}, 7, passa };
        trains[11] = { "Alektrichka-611", "Тула", "Серпухов", {13,20}, {14,35},
6, passa };
```

```
        trains[12] = { "Lastochka-787", "Санкт-Петербург", "Выборг", {9,30}, {10,45}, 5, passa };
        trains[13] = { "RuskealskiyExpress", "Санкт-Петербург", "Сортавала", {18,30}, {22,50}, 7, passa };
        trains[14] = { "Kontenerniy-78", "Владивосток", "Хабаровск", {12,15}, {8,40}, 71, tovar };
        trains[15] = { "Pskovitianka", "Москва", "Псков", {20,23}, {8,30}, 9, passa };
        trains[16] = { "Pyteec-2", "Депо Орехово", "Депо Железнодорожная", {9,0}, {11,30}, 1, remont };
        trains[17] = { "Yantarniy", "Калининград", "Зеленоградск", {8,40}, {9,15}, 4, passa };
        trains[18] = { "Express-317", "Ростов-на-Дону", "Краснодар", {15,20}, {18,50}, 11, passa };
        trains[19] = { "Kavkaz", "Москва", "Кисловодск", {8,25}, {9,40}, 13, passa };
        return trains;
    }

    void printTrain(const train& t) {
        cout << "Маршрут: " << t.marshryt << endl;
        cout << "Отправление: " << t.pynkt_otpav << " в ";
        cout << (t.time_otprav.hour < 10 ? "0" : "") << t.time_otprav.hour << ":";
        cout << (t.time_otprav.minuts < 10 ? "0" : "") << t.time_otprav.minuts << endl;
        cout << "Прибытие: " << t.pynkt_pribit << " в ";
        cout << (t.time_pribit.hour < 10 ? "0" : "") << t.time_pribit.hour << ":";
        cout << (t.time_pribit.minuts < 10 ? "0" : "") << t.time_pribit.minuts << endl;
        cout << "Вагонов: " << t.count_vagon << endl;
```

```
    cout << "Тип: " << tipname[t.tip_train] << endl;
}

//Вывести все данные по конкретному электропоезду.
void display() {

    char name[30];

    cout << "\nВведите название маршрута: ";
    cin >> name;

    for (int t = 0; t < SIZE; t++) {
        if (!strcmp(name, trains[t].marshryt)) {
            printTrain(trains[t]);
            return;
        }
    }
    cout << "Маршрут не найден.\n\n";
}

//Реализовать функцию изменения данных о маршруте.
void input(int i) {

    int tip_t;

    cout << "Название маршрута: ";
    cin >> trains[i].marshryt;

    cout << "Пункт отправления: ";
    cin >> trains[i].pynkt_otpav;
```

```
    cout << "Пункт прибытия: ";
    cin >> trains[i].pynkt_pribit;

    cout << "Время отправления (час минуты): ";
    cin >> trains[i].time_otprav.hour >> trains[i].time_otprav.minuts;

    cout << "Время прибытия (час минуты): ";
    cin >> trains[i].time_pribit.hour >> trains[i].time_pribit.minuts;

    cout << "Количество вагонов: ";
    cin >> trains[i].count_vagon;

    cout << "Тип поезда (0-товарный, 1-пассажирский, 2-ремонтный): ";
    cin >> tip_t;
    trains[i].tip_train = (tip)tip_t;
}

void update() {

    int i;
    char name[30];

    cout << "\nВведите наименование маршрута: ";
    cin >> name;

    for (i = 0; i < SIZE; i++)
        if (!strcmp(name, trains[i].marshryt)) break;

    if (i == SIZE) {
```

```
        cout << "Маршрут не найден.\n";
        return;
    }

    cout << "Введите новую информацию о маршруте";
    input(i);

    cout << "\nИнформация успешно обновлена!\n";
    printTrain(trains[i]);
}
//Реализация четния типа вагона из текстового файла (0, 1, 2)
void read_file(const char* filename) {

    char marshryt[30];
    bool found;
    int tip_value;

    ifstream fin;
    fin.open("Lab_10.txt");

    if (!fin.is_open()) {
        cout << "Ну удалось открыть файл!" << endl;
        return;
    }

    while (fin >> marshryt >> tip_value) {
        found = false;
        for (int i = 0; i < SIZE; i++) {
            if (strcmp(marshryt, trains[i].marshryt) == 0) {
                trains[i].tip_train = (tip)tip_value;
```

```
                found = true;
                break;
            }
        }
        if (!found) {
            cout << "Маршрут '" << marshryt << "' не найден" << endl;
        }
    }

    fin.close();
    cout << "Данные о типах поездов загружены из файла" << endl;
}

// Подпрограмма для записи массива структур в бинарный файл
void save_binfile() {
    ofstream out("Lab_10.bin", ios::binary | ios::out);
    out.write((char*)trains, sizeof(train) * SIZE);
    out.close();
    cout << "\nДанные успешно сохранены в бинарный файл " <<
"Lab_10.bin" << endl;
}

// Подпрограмма для чтения массива структур из бинарного файла
void load_binfile() {
    ifstream in("trains.bin", ios::binary | ios::in);
    in.read((char*)trains, sizeof(train) * SIZE);
    in.close();
}
```

//Реализовать вывод отфильтрованных данных в виде оберточной функции.

```
void obertka(const train* arr, int size) {
    for (int t = 0; t < size; t++) {
        printTrain(arr[t]);
        cout << "\n";
    }
}
```

//Переписать из исходного массива в другой массив, информацию только о тех пассажирских маршрутах, что идут в туристические города.

//Затем новый массив отсортировать по пункту отправления вагонов.

```
int tyrist_train(train* result) {
    const char* tyrist[] = { "Суздаль", "Ялта", "Сочи", "Петергоф", "Казань", "Владимир", "Выборг", "Сортавала(Рускеала)", "Псков", "Зеленоградск", "Кисловодск" };
    train nova[20] = {};
    int nova_count = 0;
    train pass_train[20] = {};
    int count_pass_train = 0;

    // Выборка только пассажирских поездов
    for (int i = 0; i < SIZE; i++) {
        if (trains[i].tip_train == passa) {
            pass_train[count_pass_train] = trains[i];
            count_pass_train++;
        }
    }

    // Переписать из исходного массива в другой массив, информацию только о тех пассажирских маршрутах, что идут в туристические города.
```

```
    for (int i = 0; i < count_pass_train; i++) {
        for (int j = 0; j < int(sizeof(tyrist) / sizeof(tyrist[0])); j++) {
            if (strcmp(pass_train[i].pynkt_pribit, tyrist[j]) == 0) {
                nova[nova_count] = pass_train[i];
                nova_count++;
            }
        }
    }

    //  Затем новый массив отсортировать по пункту отправления вагонов.
    for (int i = 0; i < nova_count - 1; i++) {
        for (int j = 0; j < nova_count - i - 1; j++) {
            if (strcmp(nova[j].pynkt_otpav, nova[j + 1].pynkt_otpav) > 0) {
                train temp = nova[j];
                nova[j] = nova[j + 1];
                nova[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < nova_count; i++) {
        result[i] = nova[i];
    }

    return nova_count;
}
//Вывести 3 самых коротких пассажирских поезда.
void short_train() {

    train pass_train[20] = {};
```

```
    int count_pass_train = 0;

    for (int i = 0; i < SIZE; i++) {
        if (trains[i].tip_train == passa) {
            pass_train[count_pass_train] = trains[i];
            count_pass_train++;
        }
    }
    // Сортировка по количеству вагонов
    for (int i = 0; i < count_pass_train; i++) {
        for (int j = 0; j < count_pass_train - 1; j++) {
            if (pass_train[j].count_vagon > pass_train[j + 1].count_vagon) {
                train temp = pass_train[j];
                pass_train[j] = pass_train[j + 1];
                pass_train[j + 1] = temp;
            }
        }
    }
    // Вывод трех самых коротких пассажирских поездов
    cout << "\n\nТри самых коротких пассажирских поезда" << endl;
    obertka(pass_train, 3);
    cout << "\n";
}

int main() {

    setlocale(LC_ALL, "");

    trains = init();
```

```
train nova[20] = {};
int nova_count = 0;

read_file("Lab_10.txt");

cout << "Поиск конкретного электропоезда";
display();

cout << "\nИзменение данных о маршруте";
update();

save_binfile();

load_binfile();
cout << "\nДанные после чтения из бинарного файла" << endl;
cout << "Название маршрута: " << trains[0].marshryt << "\n";
cout << "Количество вагонов: " << trains[0].count_vagon << "\n";
cout << "Пункт отправления: " << trains[0].pynkt_otpav << "\n";
cout << "Пункт прибытия: " << trains[0].pynkt_pribit << "\n";
cout << "Тип поезда: " << tipname[trains[0].tip_train] << "\n";

cout << "\nТуристические поезда";
nova_count = tyrist_train(nova);
cout << "\nОтсортированный список пассажирских поездов, следующих в туристические города\n";
obertka(nova, nova_count);

short_train();

delete[] trains;
```

```
    return 0;

}
```

ВЫВОД ПРОГРАММЫ

```
Данные о типах поездов загружены из файла
Поиск конкретного электропоезда
Введите название маршрута: Lastochka-701
Маршрут: Lastochka-701
Отправление: Москва в 03:45
Прибытие: Суздаль в 11:40
Вагонов: 8
Тип: пассажирский

Изменение данных о маршруте
Введите наименование маршрута: kjbh
Маршрут не найден.

Данные успешно сохранены в бинарный файл Lab_10.bin

Данные после чтения из бинарного файла
Название маршрута: Lastochka-701
Количество вагонов: 8
Пункт отправления: Москва
Пункт прибытия: Суздаль
Тип поезда: пассажирский

Туристические поезда
Отсортированный список пассажирских поездов, следующих в туристические города
Маршрут: Lastochka-803
Отправление: Адлер в 08:05
Прибытие: Сочи в 08:45
Вагонов: 5
Тип: пассажирский

Маршрут: Yantarniy
Отправление: Калининград в 08:40
Прибытие: Зеленоградск в 09:15
Вагонов: 4
Тип: пассажирский

Маршрут: Lastochka-701
Отправление: Москва в 03:45
Прибытие: Суздаль в 11:40
Вагонов: 8
Тип: пассажирский

Маршрут: Tavria
Отправление: Москва в 17:20
Прибытие: Ялта в 10:15
Вагонов: 14
```

```
Маршрут: Tatarstan
Отправление: Москва в 19:08
Прибытие: Казань в 06:30
Вагонов: 12
Тип: пассажирский

Маршрут: VladimirskiyExpress
Отправление: Москва в 07:45
Прибытие: Владимир в 09:30
Вагонов: 7
Тип: пассажирский

Маршрут: Pskovitianka
Отправление: Москва в 20:23
Прибытие: Псков в 08:30
Вагонов: 9
Тип: пассажирский

Маршрут: Kavkaz
Отправление: Москва в 08:25
Прибытие: Кисловодск в 09:40
Вагонов: 13
Тип: пассажирский

Маршрут: Kometa
Отправление: Санкт-Петербург в 10:00
Прибытие: Петергоф в 10:45
Вагонов: 6
Тип: пассажирский

Маршрут: Lastochka-787
Отправление: Санкт-Петербург в 09:30
Прибытие: Выборг в 10:45
Вагонов: 5
Тип: пассажирский
```

```
Три самых коротких пассажирских поезда
Маршрут: Yantarniy
Отправление: Калининград в 08:40
Прибытие: Зеленоградск в 09:15
Вагонов: 4
Тип: пассажирский

Маршрут: Lastochka-803
Отправление: Адлер в 08:05
Прибытие: Сочи в 08:45
Вагонов: 5
Тип: пассажирский

Маршрут: Lastochka-787
Отправление: Санкт-Петербург в 09:30
Прибытие: Выборг в 10:45
Вагонов: 5
Тип: пассажирский



C:\Users\karin\source\repos\Lab_10\x64\Debug\Lab_10.exe
Чтобы автоматически закрывать консоль при остановке отла
и остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

РЕЗУЛЬТАТ ЗАПИСИ СТРУКТУРЫ В БИНАРНЫЙ ФАЙЛ

```
?????1        ???            ???u            - 
 ( ▯ ▯ ??????         ??????         ???            ?▯
 ▯ ▯ -   ?????2         ???????a       ??u             ▯ ▯ ▯ -
 ▯ ???            ???            ??              ▯ ▯
 ▯ ▯ ▯ ???            ??????         ??????e         ▯   ▯ ▯ ▯ ▯ ??????3        ???            ??             ▯ ▯ ▯ - ▯
▯ ???            ???????a       ????

 - ▯ ▯ ?????          ?????e         ??????         ?▯ ( ▯ ▯ <   ????n          ???            ???            ?▯ ▯ ▯ ▯
 ▯ ?????          ???            ???????a       ?▯ ▯ ▯ ▯
 ▯ ?????????s     ???            ????           ?  -     ▯   ▯ ???????1       ??             ????           ?
 ▯ ▯ # ▯ ▯ ??????7        ???????a       ???            ?        ▯
 - ▯ ▯ ?????????      ???????a       ????a           ▯ ▯ ▯ 2   ▯ ???????        ?????e         ????e
 ▯ ▯ ( G   ??????         ???            ??a            ?▯ ▯ ▯ ▯         ▯ ????           ??????         ??????????     ?
 ▯ ▯ ▯ ????y          ?????a         ??????          ▯ (         ▯ ▯ ▯ ?????7         ???????        ?????          ?▯ ▯ ▯ 2
 ▯ ???            ???            ?????          ?▯ ▯     (
 ▯
```

Строка 1, столбец 1 | 1 160 символов | Обычный текст | 110% | Unix (LF) | ANSI

ФАЙЛ Lab_10.txt, где 0 – товарный, 1- пассажирский, 2 - ремонтный

```
Lastochka-701 1
Gryzovoy-115 0
Express-202 1
Tavria 1
Tex-09 2
Lastochka-803 1
Kometa 1
Ygolniy-44 0
Tatarstan 1
Sapsan-761 1
VladimirskiyExpress 1
Alektrichka-611 1
Lastochka-787 1
RuskealskiyExpress 1
Kontenerniy-78 0
Pskovitianka 1
Pyteec-2 2
Yantarniy 1
Express-317 1
Kavkaz 1
```